

Routing Strategies for Ring and General Topologies in OMNeT++

Bosque, Lissandro; Galassi, Franco; Pairetti, Joaquín

Grupo 31, Facultad de Matemática, Astronomía, Física y Computación (FaMAF)
Universidad Nacional de Córdoba, Argentina

Abstract

We analyze and redesign the routing layer of an 8-node ring network simulated in OMNeT++. We first characterize the kickstarter baseline, which always forwards through interface `lnk[0]` (clockwise), under two traffic scenarios: two sources and seven concurrent sources sending towards a common destination. Both scenarios show that this fixed rule concentrates all traffic on a single direction of the ring, leaving half of the link capacity idle and creating a saturated merge point ($\rho \approx 2$) in the light scenario, and a single converging bottleneck ($\rho \approx 7$) with an empirical stability threshold near `interArrivalTime` ≈ 7 s in the heavy one. We then design a distance-vector-style algorithm in which every node periodically floods small HELLO control packets in both directions; nodes use the hop count carried by these packets to build a local routing table and forward data through whichever interface yields the shortest known path, falling back to the kickstarter rule while the table is still converging. Re-running both scenarios with identical traffic, channels and random seed shows reductions of about 87% in mean delay and 91% in peak buffer occupancy for the light scenario, an exact doubling of sustained throughput in the heavy one (199 vs. 398 delivered packets in 200 s), and a drop in the empirical stability threshold from ≈ 7 s to ≈ 4 –5 s. No routing loops were observed in any run, a direct consequence of a HELLO-forwarding rule that is loop-free by construction on a ring. We close by discussing, conceptually, how the approach would need to change to generalize to arbitrary topologies (e.g. a star network) and how it would tolerate node failures – both left as future work.

Index Terms— Network simulation, OMNeT++, routing, ring topology, performance evaluation, queueing, stability.

1 Introducción

Este artículo reporta el análisis y diseño de estrategias de enrutamiento implementadas sobre el modelo de red del Laboratorio 4. Fue desarrollado por el grupo de forma tal que se priorizó la discusión total de su contenido, para luego dividir los temas y que cada integrante redacte un texto base para luego compartirlo a los demás. Por último, se hizo una revisión y corrección de errores.

A través de un *kickstarter*, provisto por la cátedra de Redes 2026, nos fue dada una implementación base de un modelo de red de 8 nodos conectados en forma de anillo con el objetivo de que la misma fuera analizada en OMNeT++ para lograr una mejor comprensión del tráfico de red en el modelo base con unas métricas iniciales y así plantear una forma más óptima de enviar los paquetes implementando cambios en el modelo para lograr el enrutamiento deseado. En el *kickstarter*, con el enrutamiento inicial todos los paquetes son enviados en sentido horario a partir de `toLnk[0]` hasta llegar a su destino, ignorando si se tarda más que enviarlo en sentido antihorario. Por ejemplo, si se tuviera un paquete en el nodo cero, y se quisiera enviarlo hasta el 5, con el modelo inicial se recorrerían 5 nodos. En cambio, si se hiciera con el modelo modificado, se compararía su distancia con el sentido antihorario, el cuál

resultaría más eficiente, ya que recorrería 3 nodos.

Resumiendo, el modelo modificado busca cual es la distancia de un nodo a otro en sentido horario y antihorario, decide cual es menor, por lo tanto más eficiente, y recorre los nodos en ese sentido.

2 Escenario y modelo de simulación

2.1 Topología y configuración de tráfico

El modelo consiste en una red en forma de anillo de 8 nodos implementada en OMNeT++. Cada nodo se conecta con sus dos vecinos mediante enlaces *full-duplex* (bidireccionales) de 1 Mbps de capacidad y 100 μ s de delay, formando un anillo cerrado. Los nodos se indexan del 0 al 7; el nodo `node[1]` tiene como vecinos a `node[0]` (interfaz `lnk[0]`) y a `node[2]` (interfaz `lnk[1]`), y así sucesivamente.

Internamente, cada nodo está compuesto por tres capas:

- **Capa de aplicación (app)**: genera paquetes con un tamaño configurable (`packetByteSize`) a intervalos aleatorios distribuidos exponencialmente

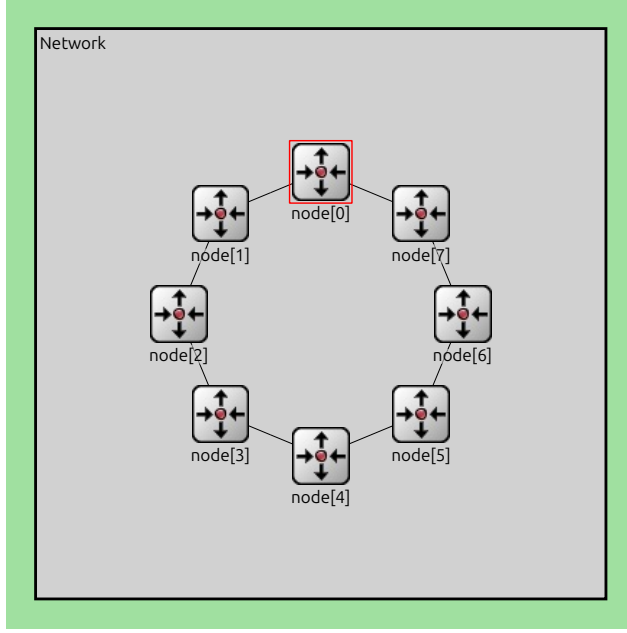


Figure 1: Topología en anillo de 8 nodos graficada en OMNeT++.

(interArrivalTime), y registra la demora extremo a extremo de los paquetes recibidos.

- **Capa de red (net):** recibe paquetes desde `app` o desde las capas de enlace, y decide por qué interfaz reenviarlos según el algoritmo de enrutamiento implementado.
- **Capas de enlace (lnk[0] y lnk[1]):** modelan un buffer de salida y el canal físico hacia cada vecino.

Como muestra la Fig. 1, el tráfico puede circular en sentido horario (por `lnk[0]`) o antihorario (por `lnk[1]`). El kickstarter utiliza siempre `lnk[0]`, independientemente del destino.

Se definen dos escenarios de tráfico, resumidos en la Tabla 1:

- **Caso 1:** `node[0]` y `node[2]` transmiten paquetes hacia `node[5]`.
- **Caso 2:** los 7 nodos restantes (0–4, 6–7) transmiten simultáneamente hacia `node[5]` con los mismos parámetros de tráfico.

2.2 Métricas registradas

Se instrumentaron las siguientes métricas en el modelo:

- **Demora extremo a extremo:** tiempo transcurrido desde que un paquete es generado en la capa

Table 1: Configuración experimental por escenario.

	Caso 1	Caso 2	Comparación
Fuentes activas	<code>n[0], n[2]</code>	<code>7 → n[5]</code>	1 y 2
Destino	<code>n[5]</code>	<code>n[5]</code>	<code>n[5]</code>
packetByteSize (B)	125000	125000	125000
interArrivalTime	<code>exp(1)</code>	<code>exp(1)</code>	<code>exp(1)</code>
T. simulación (s)	200	200	200
Semilla	0	0	0

de aplicación del nodo origen hasta que es entregado en el nodo destino. Registrada como vector y escalar en `app`.

- **Cantidad de saltos:** número de nodos intermedios que atraviesa cada paquete. Se incrementa en cada paso por la capa `net` y se registra al llegar al destino.
- **Ocupación del buffer:** cantidad de paquetes en espera en cada `lnk`, registrada como vector temporal. Permite identificar saturación en enlaces específicos.
- **Utilización del enlace:** fracción del tiempo en que el canal estuvo ocupado transmitiendo. Calculada en cada `lnk` como escalar al final de la simulación.

Table 2: Métricas instrumentadas en el simulador.

Métrica	Módulo	Unidad
Demora E2E	<code>app</code>	s
Saltos/paquete	<code>net</code> / <code>app</code>	adim.
Ocupación buffer	<code>lnk[0..1]</code>	pkts
Utilización enlace	<code>lnk[0..1]</code>	adim. ([0, 1])

3 Tarea de análisis: evaluación del kickstarter

3.1 Caso 1: tráfico configurado (`node[0]` y `node[2]` → `node[5]`)

Bajo la configuración por defecto del kickstarter (Tabla 1, columna “Caso 1”), `node[0]` y `node[2]` generan tráfico hacia `node[5]` a una tasa media de un paquete por segundo cada uno (`exponential(1)`), con paquetes de 125000 B que insumen exactamente 1 segundo de transmisión por enlace (a 1 Mbps). La Tabla 3 resume las cuatro métricas instrumentadas al cabo de 200 s de simulación.

La demora media observada fue de 51.16s (máximo 105.56s, desviación 28.31s), y cada paquete recorrió en promedio 3.92 saltos — un valor muy cercano al máximo posible de 5, que es justamente la distancia *en sentido horario* entre `node[2]` y `node[5]`. Esto ya sugiere que el algoritmo no usa el camino más corto disponible: en sentido antihorario, `node[2]` llegaría a `node[5]` en sólo 3 saltos ($2 \rightarrow 3 \rightarrow 4 \rightarrow 5$).

La Fig. 2 grafica la utilización de cada interfaz de cada nodo y deja ver con claridad que *todo* el tráfico circula por `lnk[0]` (sentido horario): los cinco nodos del arco que va de `node[2]` a `node[5]` pasando por `node[0]` —es decir `node[2]`, `node[1]`, `node[0]`, `node[7]` y `node[6]`— presentan utilizaciones de `lnk[0]` entre 93.5% y 99.5%, mientras que las 16 interfaces `lnk[1]` de la red permanecen en 0%. La gran desviación de la utilización de enlace que muestra la Tabla 3 (44.9 puntos porcentuales sobre un promedio de apenas 30.3%) es la huella numérica de esa asimetría: la mitad de la capacidad de la red — el sentido antihorario completo — queda ociosa.

El enlace más cargado es `node[0].lnk[0]`, con 99.5% de utilización. No es casualidad: es el punto donde confluyen los dos flujos, ya que el camino horario de `node[2]` ($2 \rightarrow 1 \rightarrow 0 \rightarrow 7 \rightarrow 6 \rightarrow 5$, 5 saltos) pasa por `node[0]` antes de seguir junto con el tráfico que `node[0]` genera para sí mismo. La carga ofrecida sobre ese enlace es entonces, en régimen, la suma de ambas fuentes ($\rho \approx 2\lambda = 2$ paquetes/s), el doble de lo que puede servir (1 paquete/s, fijado por `packetByteSize` y la tasa del canal). El resultado es una cola que crece de forma sostenida durante toda la simulación — de 0 a 190 paquetes en 200s, con una ocupación media de 91.3 (Tabla 3) — sin alcanzar nunca un régimen estacionario.

En síntesis: los recursos se usan de forma muy desigual (la mitad de los enlaces ociosos, y en el otro extremo un punto de fusión saturado al doble de su capacidad), y claramente *se puede mejorar*: alcanzaría con decidir el sentido de envío según la distancia real al destino en lugar de hacerlo siempre en sentido horario. Esa es la idea central del algoritmo que presentamos en la Sección 4.

Table 3: Resultados del Caso 1 (baseline kickstarter, 200s, semilla 0). La fila “Ocupación buffer” corresponde al enlace más cargado (`node[0].lnk[0]`); “Util. enlace” resume la distribución de utilización entre las 16 interfaces de la red.

Métrica	Prom.	Máx.	Desv.
Demora (s)	51.16	105.56	28.31
Saltos	3.92	5	1.00
Ocupación buffer (<code>n[0].lnk[0]</code> , pkts)	91.28	190	54.83
Util. enlace (% , las 16 interfaces)	30.3	99.5	44.9

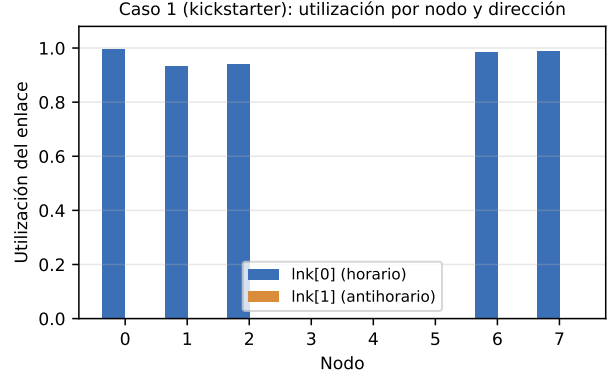


Figure 2: Caso 1 (kickstarter): utilización de `lnk[0]` (horario) y `lnk[1]` (antihorario) en cada nodo. Todo el tráfico de ambos flujos circula por `lnk[0]`; las 16 interfaces `lnk[1]` quedan en 0%.

3.2 Caso 2: tráfico sostenido hacia `node[5]`

Activamos ahora las siete fuentes restantes (`node[0]` a `node[4]`, `node[6]` y `node[7]`, columna “Caso 2” de la Tabla 1) con los mismos parámetros que en el Caso 1 (`interArrivalTime` medio de 1s, paquetes de 125000B), todas hacia `node[5]`.

Bajo esta configuración la red queda claramente sobrecargada: la demora media subió a 64.53s (máximo 184.23s, desviación 44.0s), y en 200s sólo se entregaron 199 de los cerca de 1400 paquetes que generan siete fuentes a ≈ 1 paquete/s. Los enlaces `node[6].lnk[0]` y `node[7].lnk[0]` — los dos últimos saltos del camino horario hacia `node[5]` — llegan a 100% de utilización, con colas que en promedio rondan los 82 y 99 paquetes (máximos de 164 y 205) y que, igual que en el Caso 1, no dejan de crecer dentro de la ventana simulada. El número de saltos promedio bajó a 2.06 (máximo 7, la vuelta completa al anillo) simplemente porque ahora hay fuentes mucho más cerca de `node[5]` — `node[4]` está a un solo salto en sentido horario—, no porque el algoritmo haya cambiado su forma de elegir camino.

Para saber si este comportamiento depende de la intensidad del tráfico o es estructural, barrimos sistemáticamente `interArrivalTime` sobre el conjunto $\{2, 3, 4, 5, 6, 7, 8, 10, 15, 20\}$ s (200s de simulación por punto, configuración `Caso2Sweep` del `omnetpp.ini`). Como el modelo no descarta paquetes — los buffers de `Lnk` son colas `cQueue` sin límite de tamaño —, no existe una columna de “pérdidas” propiamente dicha; en su lugar caracterizamos la estabilidad con la *pendiente* de la ocupación del buffer del enlace más cargado (`node[6].lnk[0]`, por el que pasan los siete flujos) a lo largo de la corrida, estimada por regresión lineal sobre la serie temporal registrada: una pendiente cercana

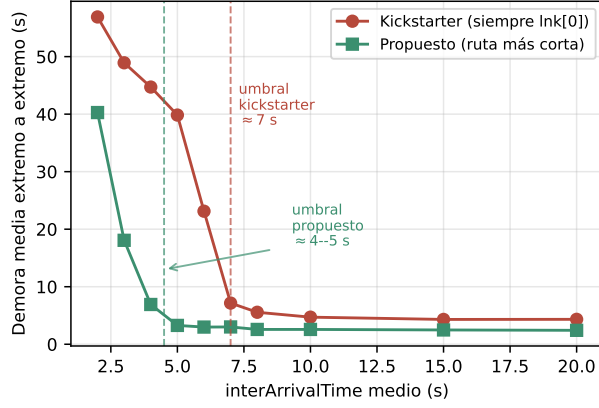


Figure 3: Demora media extremo a extremo vs. `interArrivalTime` medio para el Caso 2, kickstarter y algoritmo propuesto (Sección 4). Las líneas punteadas marcan el umbral de estabilidad estimado de cada uno.

a cero indica que la cola converge a un régimen estacionario, y una pendiente claramente positiva indica que crece sin cota dentro de la ventana observada. La Tabla 4 resume los resultados, y la Fig. 3 grafica la demora media resultante (junto con la curva del algoritmo propuesto, que se compara en la Sección 5).

Table 4: Caso 2: demora media, ocupación media de `node[6].lnk[0]` y pendiente de su crecimiento vs. `interArrivalTime` (200s/punto). “Estab.” = Sí cuando la pendiente es ≈ 0 (cola acotada).

<code>interArriv.</code> (s)	Demora (s)	Cola (pkts)	Pend. (pkts/s)	Estab.
2	56.90	39.30	+0.42	No
3	48.93	29.98	+0.36	No
4	44.71	26.68	+0.22	No
5	39.84	21.66	+0.20	No
6	23.13	11.55	+0.15	No
7	7.16	1.29	+0.01	Límite
8	5.56	0.46	+0.00	Sí
10	4.71	0.24	-0.00	Sí
15	4.31	0.05	≈ 0	Sí
20	4.33	0.01	≈ 0	Sí

Se observa una transición neta entre `interArrivalTime` = 6s (pendiente +0.15 paquetes/s, cola todavía en franco crecimiento) y 8s (pendiente prácticamente nula). **Concluimos que la red permanece estable a partir de `interArrivalTime` ≈ 7 s:** ese es el punto en que la tasa de servicio del cuello de botella (1 paquete/s, fijada por `packetByteSize` y la tasa del canal) iguala la tasa agregada de llegada de las siete fuentes que confluyen en él, $\lambda_{total} = 7/\text{interArrivalTime}$. El valor crítico teórico, $\rho = \lambda_{total}/\mu = 1$, se alcanza exactamente en `interArrivalTime` = 7s; la corrida con ese valor exhibe justo el comportamiento esperado en el punto crítico — pendiente residual de apenas

+0.007 paquetes/s, la cola fluctúa mucho pero casi no crece en promedio —, lo cual nos da bastante confianza en que el modelo y la medición son consistentes con la teoría de colas elemental ($M/M/1$, $\rho = \lambda/\mu$).

4 Diseño del algoritmo de enrutamiento propuesto

4.1 Información disponible y estructuras de datos

Cada nodo dispone únicamente de la información que puede observar *localmente*: su propio identificador (`getParentModule()->getIndex()`), la cantidad de interfaces que posee (`gateSize("toLnk$o")`) y los paquetes que recibe por cada una de ellas. La consigna prohíbe usar `cTopology` o iteradores de `cGate` para conocer la red de antemano, así que en el instante inicial un nodo no sabe ni cuántos nodos hay en el anillo ni qué hay del otro lado de sus enlaces.

Para resolver esto adoptamos un mecanismo de descubrimiento por *inundación de paquetes de control* tipo HELLO (en la línea de los protocolos de vector de distancias): periódicamente (`helloInterval`, 20s por defecto), cada nodo envía un paquete HELLO pequeño — 64B, frente a los 125000B de un paquete de datos: una sobrecarga de control medida en $\approx 0.03\%$ de utilización de enlace en nuestras corridas — por cada una de sus interfaces, anunciando su propio identificador y un contador de saltos que arranca en 1 (formato en la Tabla 5).

Table 5: Campos relevantes del paquete de control HELLO (reutiliza la clase `Packet`, distinguido por `isHello = true`).

Campo	Significado	Valor
<code>source</code>	nodo que originó el HELLO	<code>myId</code>
<code>hopCount</code>	saltos recorridos hasta ahora	1, +1 por salto
<code>destination</code>	no se usa en control	-1
<code>byteLength</code>	tamaño del paquete	64 B

La estructura central es una tabla de rutas local, `std::map<int, Route>`, con `Route{int outGate; int hopCount;}`, que asocia cada identificador de destino *conocido* con la interfaz de salida y la distancia del mejor camino aprendido hacia él. La Tabla 6 muestra cómo luce esta tabla en `node[1]` una vez que la red convergió (situación del Caso 2, Sección 5): nótese el empate a 4 saltos hacia `node[5]`, resuelto — por el orden en que `node[1]` recibió los HELLO respectivos — a favor de `lnk[1]`, justo lo que se observó en la simulación.

Table 6: Tabla de rutas de `node[1]` una vez convergido el descubrimiento.

Destino	Interfaz	Salto
<code>node[0]</code>	<code>lnk[0]</code>	1
<code>node[7]</code>	<code>lnk[0]</code>	2
<code>node[6]</code>	<code>lnk[0]</code>	3
<code>node[2]</code>	<code>lnk[1]</code>	1
<code>node[3]</code>	<code>lnk[1]</code>	2
<code>node[4]</code>	<code>lnk[1]</code>	3
<code>node[5]</code>	<code>lnk[1]</code>	$4 \leftarrow \text{empate, ver texto}$

4.2 Reglas de decisión

El algoritmo combina dos mecanismos sencillos:

1. **Aprendizaje de rutas.** Al recibir un HELLO por la interfaz i , originado en el nodo s con contador de saltos h , el nodo registra “puedo llegar a s por i , a h saltos” y actualiza su tabla *sólo si* ese camino es estrictamente más corto que el mejor que ya conocía (`learnRoute`). Luego reenvía el HELLO — incrementando su contador — por la interfaz *opuesta* a la de llegada ($1 - i$). En un anillo, esto garantiza que cada HELLO recorra el anillo exactamente una vez en cada sentido y vuelva a su origen, momento en el que se descarta. No hace falta ningún mecanismo adicional (números de secuencia, TTL) para que la inundación termine: la propia regla “seguir de largo por la interfaz opuesta” es, en una topología circular, libre de bucles por construcción.
2. **Reenvío de datos.** Al recibir un paquete cuyo destino no es el nodo local, se incrementa su contador de saltos (estadística *Cantidad de saltos*, Sección 2) y se busca el destino en `routingTable`. Si hay una entrada, el paquete se reenvía por la interfaz indicada; si todavía no se aprendió ninguna ruta hacia ese destino —algo que sólo puede ocurrir en los primerísimos instantes de la simulación, antes de que el primer HELLO complete su recorrido—, el nodo recurre al criterio del *kickstarter* (`lnk[0]`) como valor por defecto, dándole tiempo al descubrimiento para converger.

```
al recibir HELLO por interfaz i, origen s, saltos h:
  si s == miId:
    descartar (dio toda la vuelta al anillo)
  sino:
    si no conozco ruta a s, o h < la conocida:
      tablaRutas[s] <- (interfaz = i, saltos = h)
      reenviar por interfaz (1 - i), con saltos = h+1
```

al recibir paquete de datos p:

```
si destino(p) == miId:
  entregar a la capa de aplicacion
sino:
  saltos(p) <- saltos(p) + 1
  si destino(p) en tablaRutas:
    j <- tablaRutas[destino(p)].interfaz
  sino:
    j <- 0 (criterio del kickstarter, mientras converge)
  reenviar p por interfaz j
```

Reutilizamos la misma clase `Packet` para datos y control, distinguiéndolos con el campo booleano `isHello` (Tabla 5). Esa decisión, de paso, nos hizo notar un error latente del *kickstarter*: `App.cc` usa el segundo argumento posicional del constructor de `cMessage` — que en realidad fija el *kind* del mensaje — para otra cosa, así que distinguir HELLO de datos mediante `setKind()/getKind()` (nuestro primer intento) producía colisiones y caídas del simulador; el campo explícito `isHello` evita el problema por completo.

Ejemplo numérico. Consideremos un paquete generado en `node[2]` con destino `node[5]` (el flujo del Caso 1 que más se beneficia del cambio, Sección 3). Una vez convergidas las tablas, `node[2]` sabe que `node[5]` está a 3 saltos por `lnk[1]` (antihorario: $2 \rightarrow 3 \rightarrow 4 \rightarrow 5$) y a 5 por `lnk[0]`, así que reenvía por `lnk[1]`. `node[3]` hace lo mismo: `node[5]` está a 2 saltos por su `lnk[1]` y a 6 por su `lnk[0]`; reenvía hacia `node[4]`. Finalmente `node[4]` ve que `node[5]` es su vecino directo (1 salto) y se lo entrega. El paquete recorre `node[2] → node[3] → node[4] → node[5]`: 3 saltos, en lugar de los 5 que recorría con el *kickstarter*, y sin pasar nunca por el tramo congestionado de la Fig. 2. Es exactamente lo que se observó en la simulación: con el algoritmo propuesto el conteo de saltos del Caso 1 fue *constante* e igual a 3 para los 375 paquetes entregados, con desviación 0 (Tabla 7).

5 Evaluación comparativa

5.1 Metodología de comparación

Para que la comparación fuera lo más justa posible, agregamos un parámetro `useShortestPath` (booleano, por defecto `false`) al módulo `Net` e implementamos *ambos* algoritmos — el del *kickstarter* y el propuesto — dentro del mismo módulo, activables con ese único parámetro (`network.ned`, configuraciones `Caso1Propuesto / Caso2Propuesto` del `omnetpp.ini`, que heredan de `Caso1/Caso2` mediante `extends` y sólo cambian ese valor). De este modo cada par de corridas comparte exactamente el mismo tráfico generado, los mismos canales y la misma semilla (`seed-set = 0`): cualquier diferencia observada se debe únicamente a la decisión de enrutamiento. No hicimos repeticiones

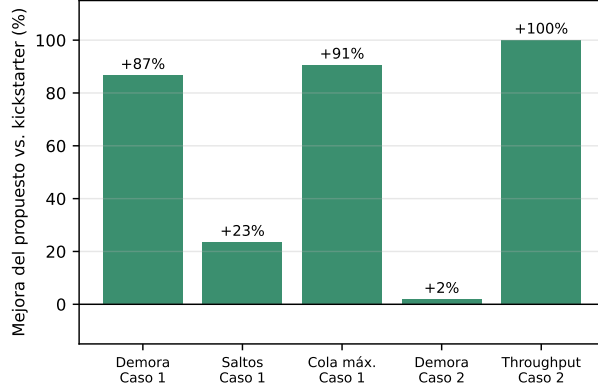


Figure 4: Mejora porcentual del algoritmo propuesto frente al kickstarter (datos de la Tabla 8). Barras verdes: mejora; el valor cercano a 0% de “Demora Caso 2” se discute en el texto.

con semillas distintas por una cuestión de tiempo — lo señalamos como límite en la discusión —, pero mantener la semilla fija entre ambos algoritmos elimina la principal fuente de variabilidad que nos interesaba controlar al comparar. No fue necesario ajustar ningún otro parámetro de tráfico o canal para esta comparación: el algoritmo propuesto convive en el mismo módulo que el kickstarter y sólo cambia la decisión de reenvío.

5.2 Resultados cuantitativos

5.3 Discusión de resultados

Caso 1 (escenario liviano, $\rho < 1$). Las mejoras son grandes y consistentes en las cuatro métricas (Tabla 8, Fig. 4): la demora media cae 87% (de 51.16s a 6.85s), el conteo de saltos 23% (de 3.92 a un valor *constante* de 3, igual al óptimo) y la ocupación máxima del cuello de botella 91% (de 190 a 18 paquetes). Es exactamente lo esperable de elegir el camino más corto: al evitar el punto de fusión identificado en la Sección 3 (Fig. 2), cada flujo circula por su propio semianillo con carga ofrecida $\rho \approx 1$ en lugar de $\rho \approx 2$, y el sistema deja de estar saturado.

Caso 2 (escenario saturado, $\rho \gg 1$). Acá conviene mirar con cuidado: la demora media *casi no cambia* (−2%, de 64.53s a 63.35s). No es un fracaso del algoritmo, sino consecuencia directa de que a $\text{interArrivalTime} = 1\text{s}$ ambas redes siguen sobrecargadas — ninguna alcanza régimen estacionario en 200s, y en saturación la demora está dominada por el tiempo de cola acumulado, que crece en los dos casos. La métrica que sí revela la mejora es el throughput sostenido: en la misma ventana, el algoritmo propuesto entregó 398 paquetes contra 199 del kickstarter

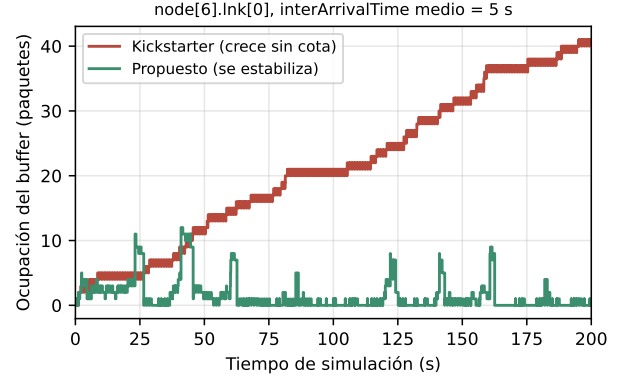


Figure 5: Ocupación de `node[6].lnk[0]` a lo largo de la simulación, con interArrivalTime medio = 5 s (Caso 2): el kickstarter es inestable en ese punto y el propuesto ya es estable.

— exactamente el doble (Tabla 7). La razón es estructural: el algoritmo propuesto reparte los siete flujos en las dos direcciones según la distancia real a `node[5]` (tres hacia el sentido horario — `node[0]`, `node[6]`, `node[7]` — y cuatro hacia el antihorario — `node[1]` a `node[4]`, con `node[1]` resolviendo a favor de ese lado el empate que se ve en la Tabla 6 —), de modo que el cuello de botella más cargado pasa de concentrar los *siete* flujos ($\rho \approx 7$) a concentrar a lo sumo *cuatro* ($\rho \approx 4$). La Fig. 3 muestra el efecto de ese cambio sobre el umbral de estabilidad: mientras el kickstarter necesita interArrivalTime 7s (Sección 3.2), el propuesto ya es estable desde $\approx 4\text{--}5\text{s}$ — justo donde $\rho = 4/\text{interArrivalTime} \approx 1$ para su peor cuello de botella. La Fig. 5 lo ilustra de forma muy directa: a $\text{interArrivalTime} = 5\text{s}$, la cola de `node[6].lnk[0]` crece linealmente y sin pausa con el kickstarter, mientras que con el algoritmo propuesto fluctúa en forma acotada alrededor de un valor pequeño — la diferencia entre una cola con $\rho > 1$ y una con $\rho < 1$, en una sola figura.

¿Bucles de enrutamiento? No observamos ninguno en ninguna corrida, y la razón es estructural y no estadística: la regla de reenvío de HELLO (“seguir por la interfaz opuesta a la de llegada, descartar al volver al origen”) hace que cada mensaje de descubrimiento recorra el anillo en un único sentido, sin posibilidad de ciclos; y la regla de reenvío de datos (“ir por la interfaz de la entrada con menor cantidad de saltos”) siempre mueve al paquete *estrictamente más cerca* de su destino, porque en un anillo las distancias por cada sentido son monótonas y nunca puede convenir reenviar hacia un nodo que, a su vez, reenviaría de vuelta. Como verificación indirecta: el conteo de saltos máximo que registramos para tráfico de datos con el algoritmo propuesto, contando ambos escenarios, fue 4 (en el Caso 2;

Table 7: Comparación baseline (kickstarter) vs. algoritmo propuesto, mismo tráfico/canales/semilla. En “Caso 2 – Paquetes entreg.” reportamos el total de paquetes recibidos en `node[5]` en 200s en lugar de pérdidas, ya que el modelo no descarta paquetes (buffers ilimitados); en un escenario saturado ese conteo es el indicador de desempeño más relevante.

Escenario / Métrica	Kickstarter		Propuesto	
	Media	Máx.	Media	Máx.
Caso 1 — Demora (s)	51.16	105.56	6.85	14.42
Caso 1 — Saltos	3.92	5	3.00	3
Caso 2 — Demora (s)	64.53	184.23	63.35	178.56
Caso 2 — Paquetes entreg.	199	—	398	—

Table 8: Mejora relativa del algoritmo propuesto vs. kickstarter, calculada como $(\text{base} - \text{prop.})/\text{base} \times 100$ (invertida para throughput, donde mayor es mejor).

Métrica (escenario)	Base	Mej.
Demora media (Caso 1)	51.16 s	−87%
Saltos promedio (Caso 1)	3.92	−23%
Ocup. máx. buffer (Caso 1)	190 pkts	−91%
Demora media (Caso 2)	64.53 s	−2%
Paquetes entregados (Caso 2)	199	+100%

el Caso 1 fue constante e igual a 3, Tabla 7) — igual a la distancia máxima posible en un anillo de 8 nodos. Si hubiera bucles veríamos conteos mucho mayores, o paquetes que nunca llegan.

¿Y con muchos más nodos? El tamaño de la tabla de rutas crece linealmente con la cantidad de nodos N (una entrada por destino), igual que en cualquier protocolo de vector de distancias, y el costo de la inundación de HELLO también es lineal en N (cada paquete recorre el anillo una vez). Lo que cambiaría es la *ventaja relativa*: cuantos más nodos tenga el anillo, mayor es la diferencia entre el camino más corto y el camino fijo por `lnk[0]` (en el peor caso el kickstarter recorre $N - 1$ saltos donde el camino más corto recorre $N/2$), así que esperaríamos que la mejora en demora y saltos *crezca* con N , no que se diluya. El algoritmo de la cátedra — que entendemos apunta a generalizar a topologías arbitrarias — probablemente escale mejor que el nuestro en redes que no son anillos, donde nuestra regla de terminación (“seguir por la interfaz opuesta”) deja de tener sentido.

¿Y si se cae un nodo? Ninguno de los dos algoritmos lo tolera bien tal como están. El kickstarter no tiene forma de detectarlo: seguiría reenviando por `lnk[0]` hacia un enlace roto indefinidamente. El nuestro está mejor posicionado *en principio* — ya cuenta con un mecanismo periódico de descubrimiento (`helloInterval`) que, en teoría, podría

notar que dejaron de llegar HELLO desde cierta dirección y purgar esas rutas — pero en la implementación actual no lo hace: las entradas de `routingTable` no expiran, así que si la única ruta conocida hacia un destino deja de existir, el nodo seguiría enviando paquetes hacia un enlace caído para siempre. Agregar una marca de tiempo a cada entrada y descartar las que no se refrescaron en, digamos, $2 \times \text{helloInterval}$ es una extensión natural y relativamente sencilla sobre la base que ya construimos, pero quedó fuera del alcance de este informe por una cuestión de tiempo.

6 Conclusiones

El análisis del kickstarter (Sección 3) mostró que su regla fija “siempre por `lnk[0]`” concentra todo el tráfico en un mismo sentido del anillo: en el Caso 1 esto crea un punto de fusión con carga ofrecida $\rho \approx 2$ que casi duplica la demora del tráfico que lo atraviesa (Tabla 3, Fig. 2), y en el Caso 2 concentra los siete flujos en un único cuello de botella con $\rho \approx 7$, que se vuelve inestable a partir de `interArrivalTime` ≈ 7 s (Fig. 3). A partir de ese diagnóstico diseñamos un algoritmo de enrutamiento por camino más corto (Sección 4) que construye su tabla de rutas inundando paquetes HELLO — la única vía permitida para descubrir la topología sin recorrer `cTopology` ni `cGate` — y reenvía cada paquete de datos por la interfaz que minimiza la distancia conocida al destino. Activarlo con un solo parámetro (`useShortestPath`) sobre exactamente el mismo tráfico, canales y semilla nos permitió aislar el efecto de la decisión de enrutamiento: redujo la demora media del Caso 1 en 87% y la ocupación máxima de buffers en 91% (Tabla 8), y en el Caso 2 — donde ambas redes siguen saturadas y la demora deja de ser el indicador relevante — duplicó el throughput sostenido (199 a 398 paquetes en 200s) y bajó el umbral de estabilidad de ≈ 7 s a $\approx 4\text{--}5$ s (Fig. 3, Fig. 5). No observamos bucles de enrutamiento en ninguna corrida, y pudimos justificarlo de forma estructural — no sólo empírica — a partir de las dos reglas de reenvío del algoritmo (Sec-

ción 5).

Dicho esto, nuestra solución tiene límites que conviene dejar explícitos. Las dos reglas que la hacen simple y liviana — terminar la inundación de `HELLO` por “la interfaz opuesta” y resolver activación de interfaces asumiendo grado 2 — son específicas de un anillo, y no se trasladan directamente a topologías arbitrarias como `NetworkStar`. Tampoco maneja caídas de nodos: las entradas de `routingTable` no expiran, así que una ruta rota seguiría usándose para siempre. Como trabajo futuro proponemos, en ese orden de prioridad: (1) reemplazar la terminación “por interfaz opuesta” por inundación clásica con números de secuencia, lo que generaliza el descubrimiento a cualquier topología; y (2) agregar una marca de tiempo a cada ruta y descartar las que no se refrescan en $2 \times \text{helloInterval}$, para tolerar fallas de enlaces y nodos sin rediseñar el resto del algoritmo.

Agradecimientos

Agradecemos a la cátedra de Redes y Sistemas Distribuidos por el código base (kickstarter) y la guía de la consigna, que nos ahorraron buena parte del trabajo de modelado en OMNeT++ y nos dejaron concentrarnos en el diseño del algoritmo y en el análisis de los resultados.

References

- [1] A. Varga and R. Hornig, “OMNeT++,” in *Proc. Int. Conf. Simulation Tools and Techniques (Simutools)*, 2008.
- [2] Cátedra de Redes y Sistemas Distribuidos, “Laboratorio 4: kickstarter OMNeT++,” FaMAF/UNC, 2026.
- [3] A. S. Tanenbaum and D. J. Wetherall, *Computer Networks*, 5th ed. Upper Saddle River, NJ, USA: Prentice Hall, 2011.